

Spring Boot - introduction

Creating a simple web application in the Spring framework

Syllabus

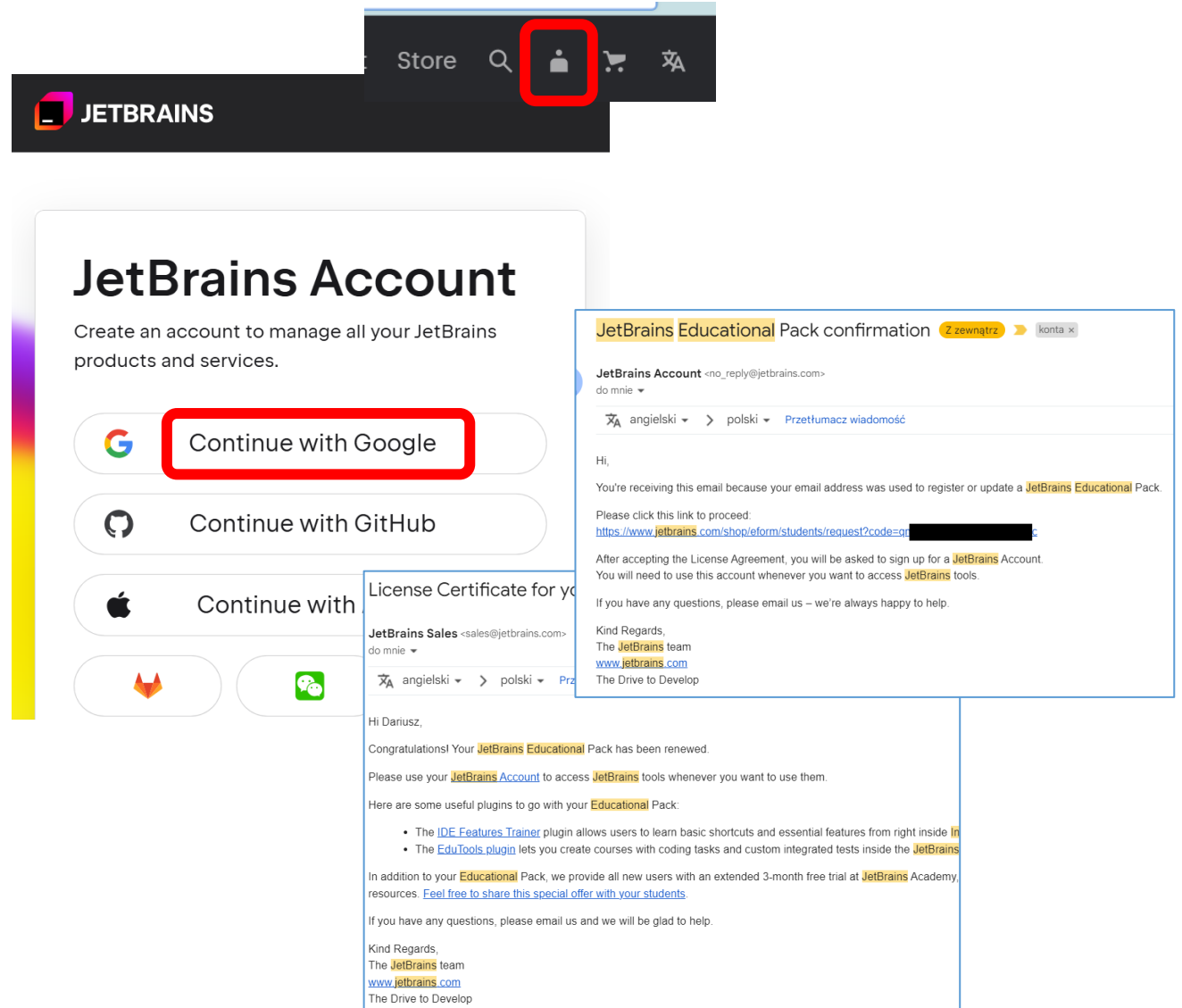
- Used IDEs
 - **IntelliJ IDEA Ultimate**
 - Eclipse
- Create a simple design
 - Spring - project generator using `start.spring.io`
 - Gradle Groovy - project dependency management
- DIP, IoC, DI
- Application `HelloWorld`
 - Web Application Class
 - Simple controller (request mapping)
 - Passing parameters to a view master model
 - View Pattern with Thymeleaf
- Application: `StudentList` with student list, additional elements:
 - Service with collection
 - Controller with GET and POST request support,
 - CRUD operations on a collection
 - View patterns for CRUD operations
 - Passing parameters from view patterns to controller actions
 - By query string
 - By URL snippet
 - By mapping the body of a POST request to a POJO object
- Fragments w Thymeleaf, layouts using fragments

Used IDEs

- The main environment presented at the lecture
 - IntelliJ IDEA Ultimate
 - After registering with a polytechnic address – free for educational purposes for one year with the possibility of renewal every year.
- Optional
 - IntelliJ IDEA Community
 - Apparently, you can configure, install plugins, etc. to work like the Ultimate version for the lecture examples
 - Eclipse with Spring Tools 4 – for Spring Boot
 - Behavior similar to the main environment
 - Other, e.g. Visual Studio Code:
 - Google: „Spring Boot in Visual Studio Code”

Creating an account on JetBrains

- Go to the <https://www.jetbrains.com/> page and select the man icon in the top menu
- Or go to the website <https://account.jetbrains.com/login>
 - Register with a **polytechnic** Google account
 - Choose an educational package
 - A letter with an activation link will be sent to the polytechnic mailbox
 - After clicking on the link, an activation confirmation email will be sent



Instalacja IntelliJ IDEA Ultimate

- Being logged in to JetBrains you can now install IntelliJ IDEA Ultimate (now: version 2025.2.4)
- Although there is information about only 30-day trial access, you should choose the Ultimate version

The image shows two overlapping screenshots from the JetBrains website. The top screenshot is the '1 Educational Pack License' page, which displays the 'JetBrains Educational Pack' logo, a 'Download' button, and a 'License ID' input field. Below this, it lists the license details: 'Licensed to: Dariusz Konieczny', 'License restriction: For educational use only', and 'Valid through: September 02, 2026'. A list of included products is shown, with 'IntelliJ IDEA Ultimate' highlighted by a red box. The bottom screenshot is the IntelliJ IDEA Ultimate download page, featuring the text 'The Leading IDE for Professional Development in Java and Kotlin' and a 'Download' button, which is also highlighted by a red box. The page also shows the 'Free 30-day trial' offer and a dropdown menu for selecting the operating system (Windows, macOS, Linux).

1 Educational Pack License

Buy new license

JetBrains Educational Pack

Download

License ID:

Licensed to: Dariusz Konieczny
License restriction: For educational use only
Valid through: September 02, 2026
Following products included:

- CLion
- DataGrip
- DataSpell
- dotCover
- dotMemory
- dotTrace
- GoLand
- IntelliJ IDEA Ultimate
- PhpStorm
- PyCharm
- ReSharper
- ReSharper C++
- ReSharper SQL support
- Rider
- RubyMine
- RustRover
- WebStorm

After downloading and installing the software, simply run it and follow the on-screen prompts to sign in with your JetBrains Account.

IntelliJ

The Leading IDE for Professional Development in Java and Kotlin

Download

Windows macOS Linux

IntelliJ IDEA Ultimate

The Leading IDE for Professional Development in Java and Kotlin

Download exe (Windows) Free 30-day trial

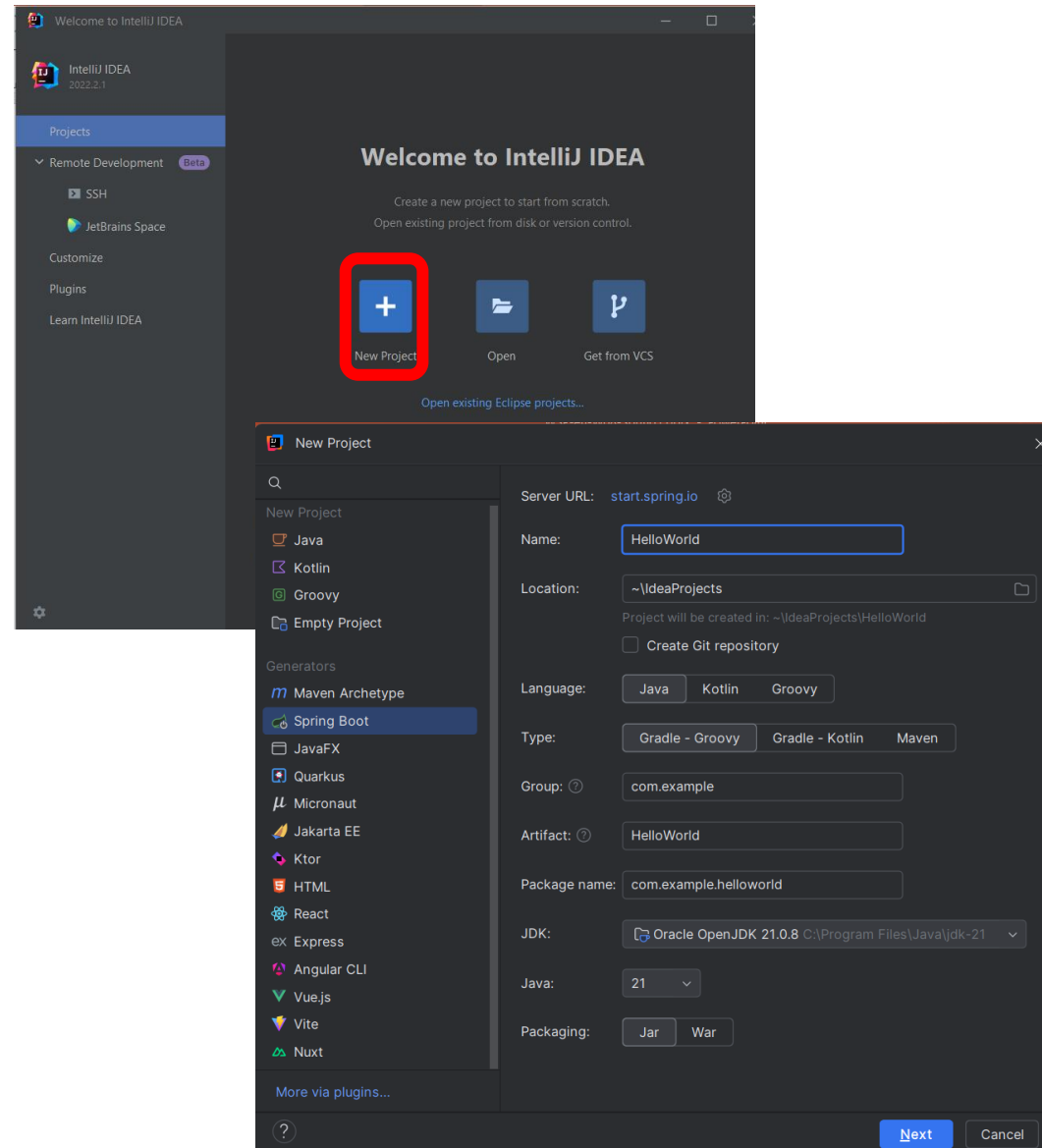
Activation of educational licenses for IntelliJ IDEA Ultimate

- **After installing** and running this version of IntelliJ, press the login button for your JetBrains account.
- After correct authorization, return to the IDE environment and press the activation button.
- You'll see an expiration date for your subscription
- You'll see an expiration date for your subscription
- Of course, this is done only when you first start this environment



Create a project 1/4

- Before you start you have to install JDK 21 from <https://www.oracle.com/java/technologies/downloads/#java21>
- Since the Ultimate version has the appropriate components of the environment installed, creating a web application project is quite simple.
- We select the plus button ("New project") to create a new project.
- To create a Spring Boot project on the left, select "Spring Boot"
- For the first simple application, we select/complete the data as in the screenshot below.
 - More details on the next slide



Create a project 2/4

- Options:

- **Server ULR:** The server used to generate the project is `start.spring.io`
- **Name:** the name of our project, e.g. HelloWorld
- **Location:** where on the local disk will be the project file. The tilde '~' for the Windows operating system stands for "C:\Users\<userName>".
- **Language:** languages based on Java virtual machine, select "Java"
- **Type:** How to manage dependencies in the project, choose Gradle-Groovy
- **Group:** In the company it should be its domain, we only need the default value "com.example"
- **Artifact:** Product name within the domain, based on **Name** populated with „HelloWorld"
- **Package name:** appropriate combination of **Group** and **Artifact** in the name of the Java package, in this case "com.example.helloworld"
- **JDK:** JDK to use by IntelliJ - choose JDK 21
- **Java:** Java version used by the compiler, choose the same version 21
 - Important 1: if you choose not equal version you can have a problem later
 - Important 2: The newest version of IntelliJ does not cooperate well with JDK 25, this is why we will use JDK 21. Probably in the future it will be fixed.
- **Packaging:** the way of packing the project, we leave "Jar".
- Move forward – button „Next"

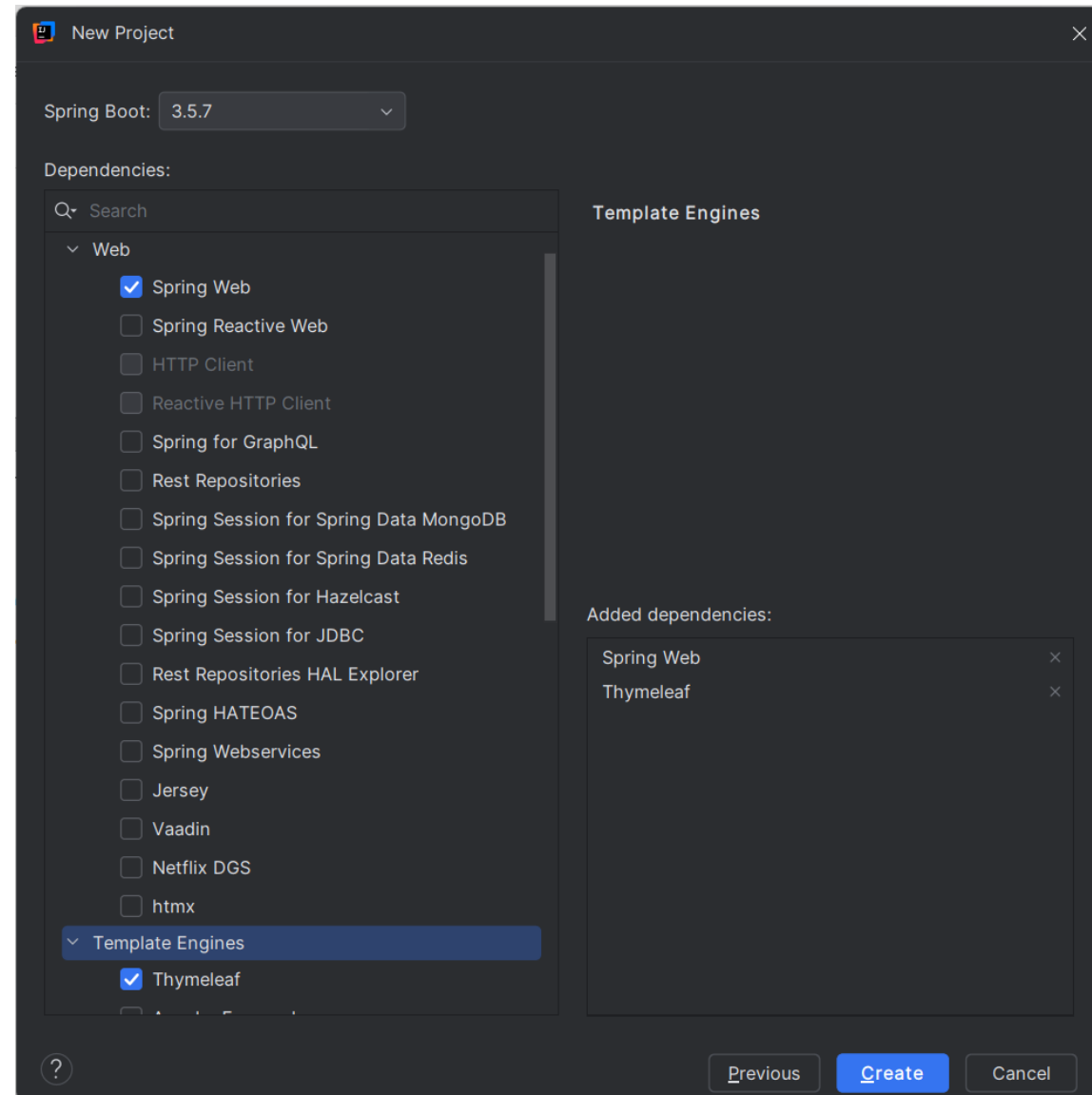
The screenshot shows the 'New Project' dialog in IntelliJ IDEA. On the left, under 'Generators', 'Spring Boot' is selected. The right panel contains the following configuration:

- Server URL: `start.spring.io`
- Name: `HelloWorld`
- Location: `~\IdeaProjects`
- Project will be created in: `~\IdeaProjects\HelloWorld`
- ☐ Create Git repository
- Language: `Java`
- Type: `Gradle - Groovy`
- Group: `com.example`
- Artifact: `HelloWorld`
- Package name: `com.example.helloworld`
- JDK: `Oracle OpenJDK 21.0.8 C:\Program Files\Java\jdk-21`
- Java: `21`
- Packaging: `Jar`

At the bottom, there is a '?' icon, a 'Next' button, and a 'Cancel' button.

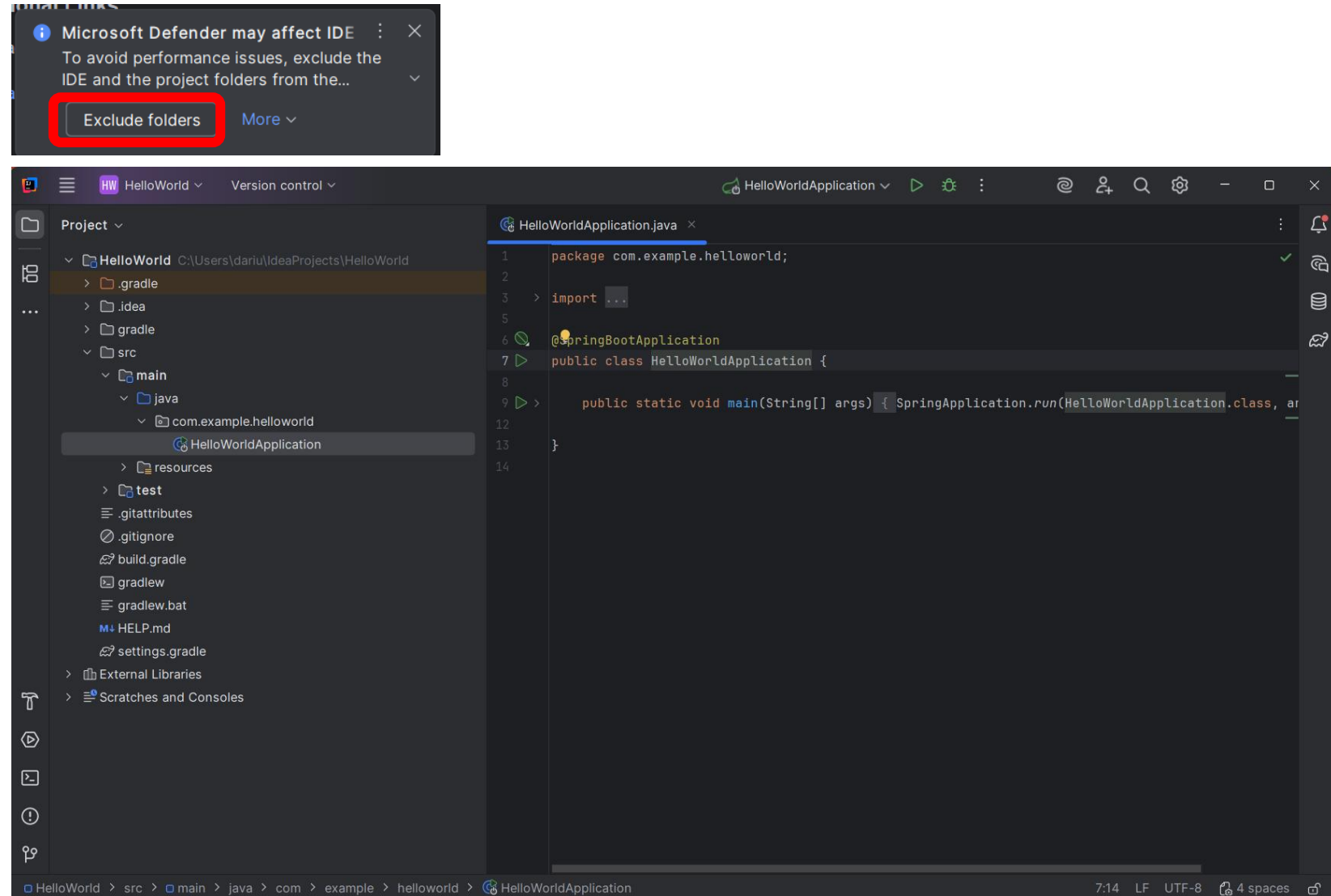
Create a project 3/4

- There are different versions of Spring Boot, we will choose the default one, i.e. 3.5.7
- You still need to select the initial dependencies
- For the simplest project, it is enough:
 - Spring Web (in the Web branch)
 - Thymeleaf (in Template Engines branch)
- Finally, we create the project with the "Create" button.
- At this point, the "Spring Initializr" plugin connects to the `start.spring.io` page, copies the information you have just set and receives the zipped project file.



Create a project 4/4

- At this point, the whole project is being prepared
 - Creates a folder skeleton
 - The files are extracted
 - configuration
 - with the main class of the Spring Boot web application
 - Other (e.g. for Gradle)
 - Then the project management system (Gradle) loads
 - It is run to retrieve the necessary dependencies
 - After some time, this process ends and you can proceed to writing the code
- It's worth to exclude the project folders from Microsoft Defender.
 - a pop-up window appears with such a proposal.
- The program can be run, but it will only show a page with an error - we need add controller and views.
- To understand how everything works together in Spring Boot, you need to understand some design concepts...



Three design concepts: DIP, IoC, DI

Dependency Inversion Principle

Inversion of Control

Dependency Injection

General rules

- What is constant in programming is ... **CHANGES**
- Components/objects should be combined in such a way that replacing one (e.g. with a new version, new library/technology, etc.) does not cause the need to change the others
 - The change may be **small** at the moment, but it causes the need to **recompile all** directly or indirectly dependent classes
- The division into **smaller**, interchangeable components also allows for **easier testing**:
 - Instead of using classes that perform certain operations that are more difficult to test (writing to the database, sending e-mails, long-term operations), you can replace them with classes-shells pretending to perform these operations
- etc.

Dependency Inversion Principle (DIP)

- Dependency Inversion Principle (DIP)
- The principle of reversing dependencies is a design pattern that tells us about writing loosely related classes:
 - **High-level** modules should **not depend** on **low-level** modules
 - **Abstractions should not** depend on **details**. It is the details that should depend on the abstraction
- Introduced by Robert C. Martin

Example of lack DIP

- The **higher-level** class **depends** on the **lower-level** class (its implementation)
- Package `dip.stage1`

```
package dip.stage1;

public class Device {
    LogWriter logWriter;

    public void notify(String message) {
        if (logWriter == null)
            logWriter = new LogWriter();
        logWriter.write(message);
    }

    public void doSomething() {
        notify("start of doSomething");
        // hard work
        notify("end of doSomething");
    }
}
```

```
package dip.stage1;

public class LogWriter {
    public void write(String message)
    {
        System.out.println("Logger: " + message);
    }
}
```

```
package dip.stage1;

public class TestOfUse {
    public static void Test(){
        Device device = new Device();
        device.doSomething();
    }
}
```

Logger: start of doSomething
Logger: end of doSomething

Inversion of Control (IoC)

- Inversion of Control (IoC), control reversal, a mechanism by which **higher-level** modules **can depend** on **abstraction** rather **than** on a **specific implementation** of a **lower-level** module.
- Abstraction/interface must be created
 - And its possible implementations
- Example in the package `dip.stage2`

```
public interface ILogNotification {  
    void notify(String message);  
}
```

```
public class LogWriter implements ILogNotification{  
  
    @Override  
    public void notify(String message) {  
        System.out.println("Logger: "+ message);  
    }  
}
```

```
public class EmailSender implements ILogNotification{  
    @Override  
    public void notify(String message) {  
        System.out.println("Sending email: "+ message);  
    }  
}
```

```
public class SMSSender implements ILogNotification{  
    @Override  
    public void notify(String message) {  
        System.out.println("Texting: "+ message);  
    }  
}
```

Solution through the use of DIP

- Fixed solution, but not entirely (dependency still exists).
 - Still, the upper level class depends on the lower level class

```
public class Device {  
    private ILogNotification logWriter;  
  
    public void notify(String message) {  
        if (logWriter == null)  
            logWriter = new LogWriter(); // still here  
        logWriter.notify(message);  
    }  
  
    public void doSomething() {  
        notify("start of doSomething");  
        // hard work  
        notify("end of doSomething");  
    }  
}
```

```
public class TestOfUse {  
    public static void Test(){  
        Device device = new Device();  
        device.doSomething();  
    }  
}
```

Logger: start of doSomething
Logger: end of doSomething

Dependency Injection (DI)

- Using Dependency Injection technique, to completely apply the DIP pattern. There are 3 ways to inject dependencies:
 - by constructor
 - by method
 - by field (property) or setter
- Spring Web has built-in (through the reflection mechanism) methods of injecting dependencies (all 3 ways)
- Assumption: Implementation of interfaces and classes implementing it as in the previous solution
 - Package `dip.stage3`

```
public interface ILogNotification {  
    void notify(String message);  
}
```

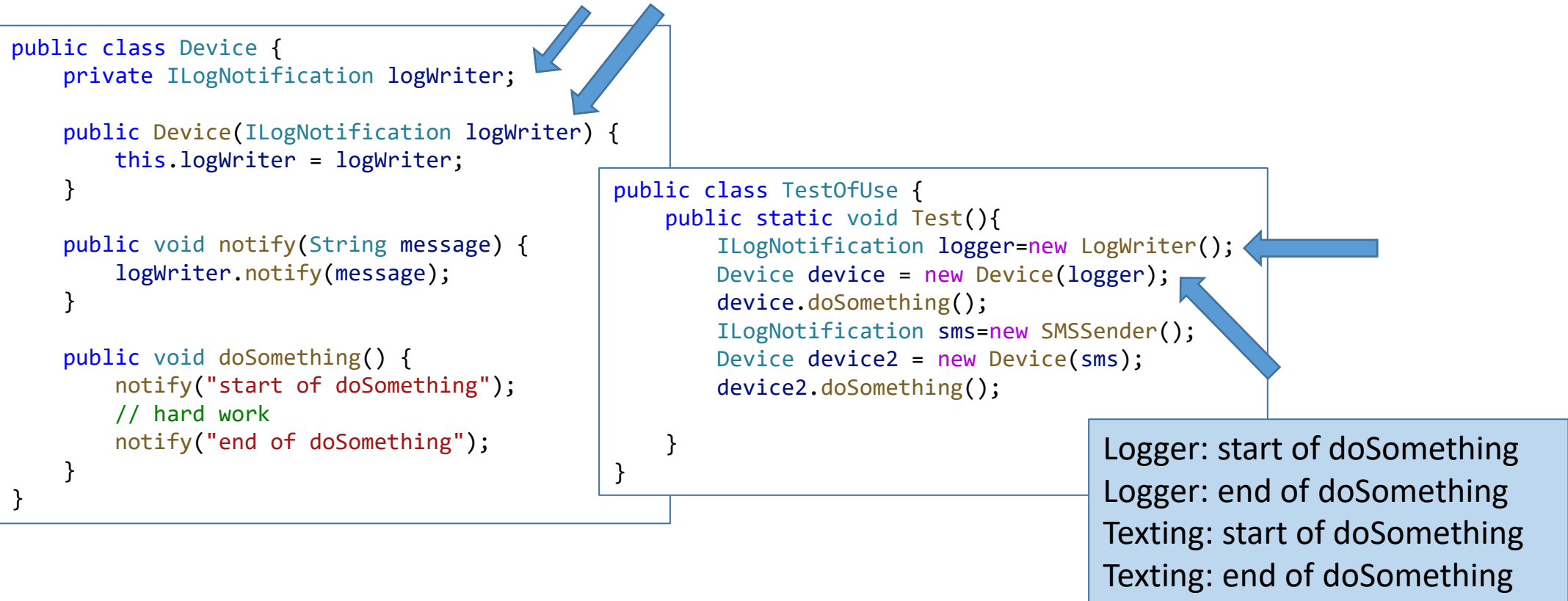
```
public class LogWriter implements ILogNotification{  
  
    @Override  
    public void notify(String message) {  
        System.out.println("Logger: " + message);  
    }  
}
```

```
public class EmailSender implements ILogNotification{  
    @Override  
    public void notify(String message) {  
        System.out.println("Sending email: " + message);  
    }  
}
```

```
public class SMSSender implements ILogNotification{  
    @Override  
    public void notify(String message) {  
        System.out.println("Texting: " + message);  
    }  
}
```

DI by constructor

- A very good solution is for the constructor to inject dependencies, especially if we know that it will never be an empty reference (**null**) and the injected class will **be used in many methods** of the higher-level class.
- Package `dip.stage3.constructor`



DI by method (method parameter)

- If you **need to change** the interface implementation of lower-level objects **during the lifetime** of a higher-level object, you can specify the implementation as a method parameter in the methods
 - Or if a lower-level object is needed **only for** the duration of **this one method**
- Package `dip.stage3.methodargument`

```
public class Device {  
  
    public void notify(ILogNotification logWriter, String message) {  
        logWriter.notify(message);  
    }  
  
    public void doSomething(ILogNotification logWriter) {  
        notify(logWriter, "start of doSomething");  
        // hard work  
        notify(logWriter, "end of doSomething");  
    }  
}
```

```
public class TestOfUse {  
    public static void Test(){  
        Device device = new Device();  
        ILogNotification logger=new LogWriter();  
        device.doSomething(logger);  
        Device device2 = new Device();  
        ILogNotification sms=new SMSSender();  
        device2.doSomething(sms);  
    }  
}
```

Logger: start of doSomething
Logger: end of doSomething
Texting: start of doSomething
Texting: end of doSomething

DI by field (property) or setter

- Instead of specifying an additional parameter each time, it is better to store the currently selected implementation in the field (properties):
 - Especially if we use the implementation more often
 - Or we often change it during the "lifetime" of a higher-level object
 - Code similar to the first version, but instead of a private field is public (or setter)
- Package `dip.stage3.field`

```
public class Device {  
  
    public ILogNotification logWriter;  
  
    public void notify(String message) {  
        logWriter.notify(message);  
    }  
  
    public void doSomething() {  
        notify("start of doSomething");  
        // hard work  
        notify("end of doSomething");  
    }  
}
```



```
public class TestOfUse {  
    public static void Test(){  
        Device device = new Device();  
        ILogNotification logger=new LogWriter();  
        device.logWriter=logger;  
        device.doSomething();  
  
        Device device2 = new Device();  
        ILogNotification sms=new SMSSender();  
        device2.logWriter=sms;  
        device2.doSomething();  
    }  
}
```



Logger: start of doSomething
Logger: end of doSomething
Texting: start of doSomething
Texting: end of doSomething

DIP, IoC, DI in Spring Web

- Spring Web is basically **all based on injection** (details will be presented at the time of use)
- By running an application class, a web server is created, **which at startup browses** (using the **reflection mechanism**) the classes (their annotations, constructors, fields and methods) that the developer created in the project.
 - It finds different types of bean (bean classes) and injection places
 - Descriptions of beans and their creation, injection can be both in the code and in the configuration files
- The **creation** of beans is carried out only **at the time of the need** for them.
 - A tree (or even a directed acyclic graph) of dependencies is then created. For example, if class A in a constructor requires an injection of a class B object, and the constructor of class B requires an injection of a class C object, the application first creates a class C object, injects it into the constructor of a class B object, and injects the newly created class object into the constructor of class A. Then, if there is injection through fields, the application will start creating the appropriate objects and injecting into the fields. When it then runs the Class A object method, a similar mechanism may occur if the injection occurs in the parameters.

Spring Framework ecosystem

- Many elements – a simple graph
- On this course - only a few components
- Assumption – ease of creating a classic application and its development
- You can still create non-classic applications by inserting your own classes "in the middle" of the finished skeleton
- This part of the lecture:
 - Spring Boot
 - Spring Web

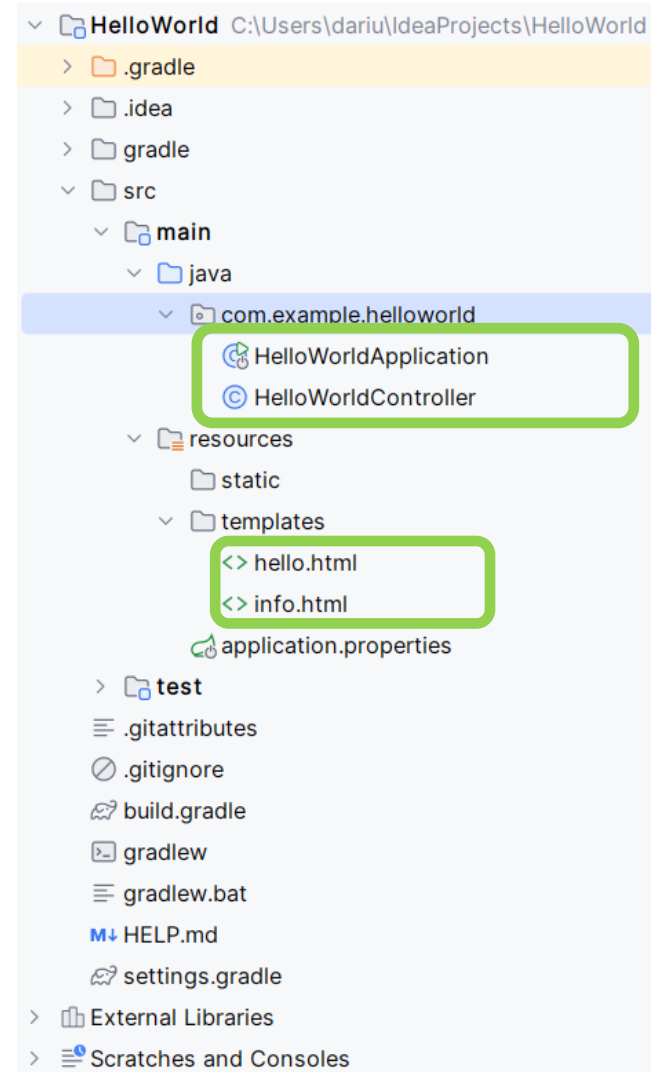


Spring Boot

- << *Spring Boot is a design part of the Spring ecosystem that introduces the concept of "**convention over configuration**". The main idea of this concept is that instead of configuring all the framework configurations yourself, Spring Boot offers a **default configuration** that you can customize as needed. In general, the result is that you **write less code** because you follow familiar **conventions** and your application differs from others in a few or small ways. So instead of writing all the configurations for each application, it is more efficient to start with the default configuration and change only what differs from the convention.>> [„Spring start Here” Laurentiu Spilca]*
- The power of Spring Boot can be seen especially in connection with Spring Initializr and Gradle (or Maven).
- Once created, the project is very easy to supplement only with business logic and user interface.
 - Alternatively, connect to a database

A simple application example

- Let's complement the HelloWorld app with a controller and views.
- Application codes should be in the `src/main/java` folder
 - And further for packages e.g. `com.example.helloworld` will be `com/example/helloworld`, but if there are no branches, then the notation with dots will remain.
 - In the above-mentioned package will be all classes of the sample application
- View patterns (simply "views" for short) should be located in the `src/main/resources/templates` folder
 - For Thymeleaf view pattern processing engine, the files should have the `.html` extension
 - You can arrange views into subfolders
 - In the case of this application, all views will be in the above mentioned folder



Controller HelloWorldController

- The HelloWorldController must have the @Controller annotation before the class definition
 - **import** org.springframework.stereotype.Controller;
 - It is a kind of bean.
 - It is a specialization in annotation @Component
 - Annotation allows automatic detection of this class when **scanning** classes.
- The **methods** (in the class) used to serve the HTTP request must have an annotation for the URL path pattern mapping.
 - Before the hello() method there is an annotation @RequestMapping("/"), that is, it is to be run for the path "/"
 - import org.springframework.web.bind.annotation.RequestMapping;
 - In this case, the method returns a **string**, which is the **name of the view file** ("hello")
 - Other types may also come back

```
@Controller
public class HelloWorldController {

    @RequestMapping("/")
    public String hello(){
        return "hello";
    }

    ...

}
```

hello.html view and effect

- Because the Thymeleaf engine is used, the ".html" extension is appended to the view name and the hello.html file is processed.
- This file has no special codes (pure HTML code), so it will be sent in its original form as a response to the request.
- Launching the browser at address `http://localhost:8080/` will show the effect of the application's operation
- Checking the content of the page source confirms that it is the same code

The image shows a development environment with three main components:

- IDE Console:** Displays the startup logs of the application. A red box highlights the port number `8080` in the log: `Tomcat started on port 8080 (http) with context path '/'`. A blue arrow points from this log entry to the browser's address bar.
- Code Editor:** Shows the `HelloWorldController.java` file. A red box highlights the 'Run' button (a green play icon) in the toolbar.
- Browser:** Displays the result of the application. The address bar shows `http://localhost:8080`. The page content shows "Hello" in a large font and "Hello World!" below it. A separate window shows the "view-source:http://localhost:8080/" page, which displays the raw HTML code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Hello screen</title>
</head>
<body>
<h1>Hello</h1>
<p>Hello World!</p>
</body>
</html>
```

information() action

- Actions (methods for handling HTTP requests) before they return the file name of a view can prepare the data needed to create such a view.
- Another method will be created in the `HelloWorldController` called `information()`.
- This data is called a **view model** (in short a **model**, although there will be more models later) and is a **hash array** with a **string key**. The **value** paired with such a key can be given of **any type**.
- One method of using a data object for a model is to inject a bean of the `Model` class into the parameters of the method
- In the `information()` action, we pass three pieces of data, two of which are an array, using the `model.addAttribute()` method to specify the **key** and its **value**.
- The action is to be called by the path `/info`
- The result is `info` referring to the `info.html` view.

```
@Controller
public class HelloWorldController {

    ...

    @RequestMapping("/info")
    public String information(Model model){
        model.addAttribute("name", "Newmann");
        model.addAttribute("kids", new String[]{"Al", "Jennifer", "Codi"});
        model.addAttribute("colors", new String[]{"color:red", "color:yellow", "color:blue"});
        return "info";
    }
}
```

model injection



View: info.html

```
<!DOCTYPE html>
<html lang="en"
  xmlns:th="http://www.thymeleaf.org en">
<head>
  <meta charset="UTF-8">
  <style>
    table, th, td {
      border: 1px solid;
    }
  </style>
  <title>Information</title>
</head>
<body>
<h1>Name: <span th:text="${name}"></span></h1>
<table>
  <thead>
    <tr>
      <td>Kid name</td>
    </tr>
  </thead>
  <tbody>
    <tr th:each="kid: ${kids}">
      <td th:text="${kid}"></td>
    </tr>
  </tbody>
</table>
<p>Some text</p>
```

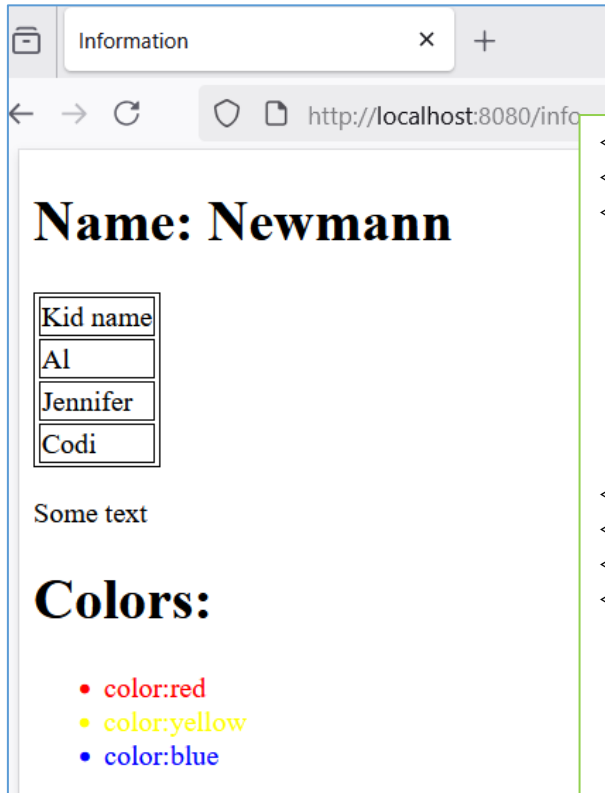
- Using **special Thymeleaf attributes** requires an appropriate namespace, i.e. appending to the element `<html>` attribute:
 - `xmlns:th="http://www.thymeleaf.org,,`
- This means that attributes preceded by the `"th:"` prefix will be given special treatment.
 - Not to be confused with `<th>` HTML elements
 - Of course, if you change the beginning of the previous line to, for example, `xmlns:thyme=...`, then use the prefix `"thyme:"`
- Template attribute, in short the attribute, `th:text` is a value that is inserted as the text content of an element inserted between the opening and closing tags of the element
 - Np. `<span th:text="${name}"></span>`
- **EL expressions** are inserted as the value of the `"th:"` attributes

```
<h1>Colors:</h1>
<ul>
  <li th:each="color: ${colors}" th:text="${color}" th:style="${color}"></li>
  <!-- <li th:each="color: ${colors}" th:text="${color}"
  th:style="${color};size:30px"></li> does not work! -->
</ul>
</body>
</html>
```

Expressions in view templates

- **Expression Language (EL)**
- *Standard Expressions* has dialects :
 - *OGNL* (Object-Graph Navigation Language) opensource Expression Language (EL) for Java. OGNL uses Java reflection mechanisms to indicate the graph of objects at the time of application startup. This allows the program to change the behavior based on the state of the object graph instead of relying on compile-time settings [Wikipedia].
 - *SpringStandard*, similar to OGNL
- Expressions of this type are enclosed in curly braces preceded by a special character: \$, *, #, @, ~. The parentheses use data from the view model, among others.
 - For example, \${name} is the value under the "name" key of the view model
- In views, expressions are enclosed in **quotation marks**.
- For example, Thymeleaf allows you to create variables iterable with respect to a collection (array) using notation with an attribute th:each:
 - `<tr th:each="kid: ${kids}">`
- **The meaning of the above notation:** for each element in the kids collection, save it into the local variable kid and use for the entire element <tr> (also in internal elements)
 - So in EL expressions there may be a kid key.
- Each attribute preceded by the prefix th: is set to the **value of the expression** (instead of some constant value)
 - For example, th:style="\${color}" will insert \${color} as the style attribute
- **Only** EL expressions must be enclosed in quotation marks:
 - For example, th:style="\${color};size:30px" generates a **run-time error** when processing the view.

Effect – browser and page source



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <style>
    table, th, td {
      border: 1px solid;
    }
  </style>
  <title>Information</title>
</head>
<body>
  <h1>Name: <span>Newmann</span></h1>
  <table>
    <thead>
      <tr>
        <td>Kid name</td>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Al</td>
      </tr>
      <tr>
        <td>Jennifer</td>
      </tr>
      <tr>
        <td>Codi</td>
      </tr>
    </tbody>
  </table>
```

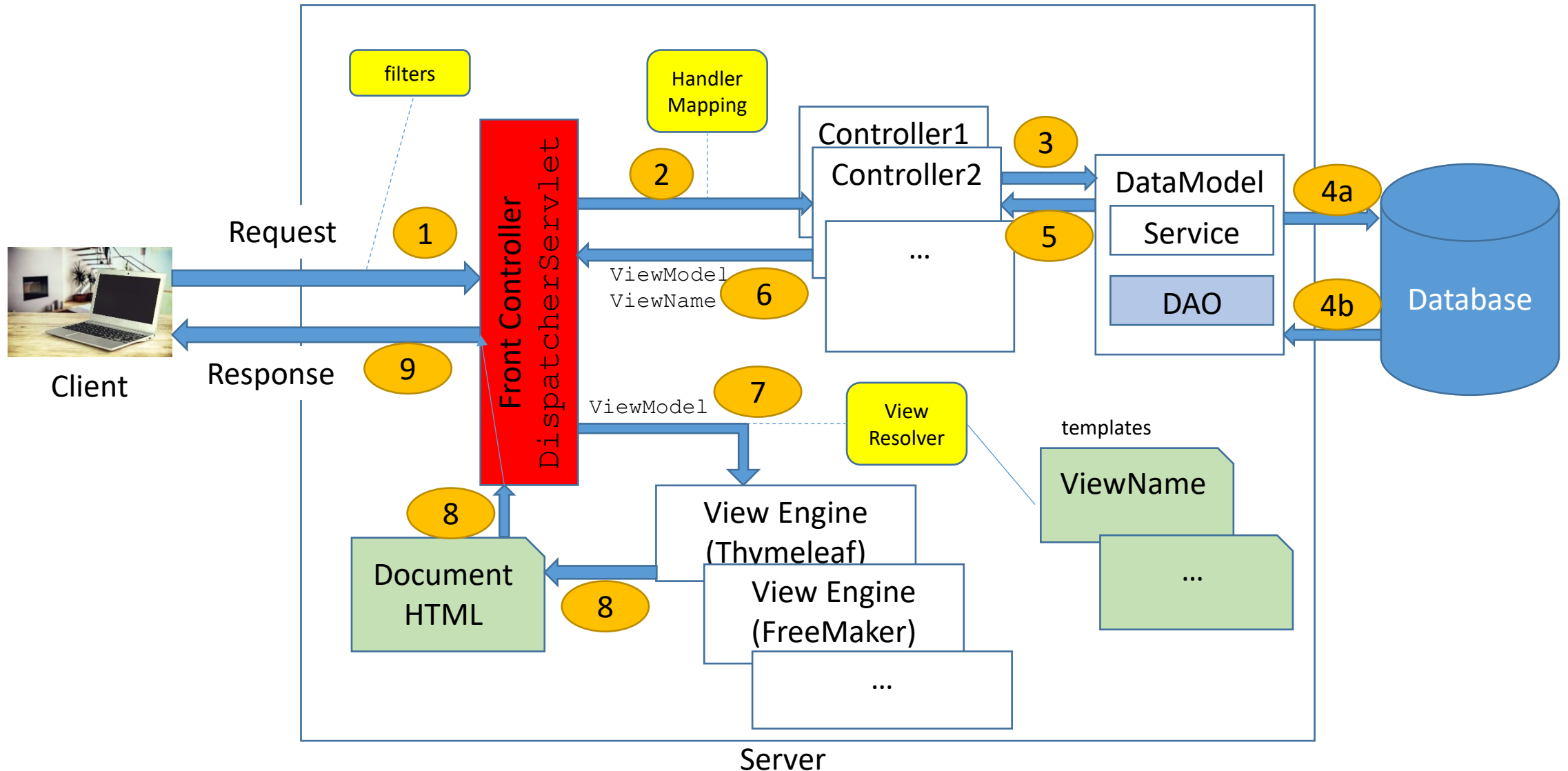
```
<h1>Colors:</h1>
<ul >
  <li style="color:red">color:red</li>
  <li style="color:yellow">color:yellow</li>
  <li style="color:blue">color:blue</li>
  <!-- <li th:text="{color}" th:style="{color};size:30px"></li>
does not work! -->
</ul>
</body>
</html>
```

- The `xmlns:th` namespace is not present in the element `<html>`
- Attributes `th`: not present
- Elements `<tr>` with different names (from the model array) inside these elements
- Elements `<li>` with different colors (from the model array) in the interior and in the `styles` attribute.

Terminology

- First there were Java **applets** (1995) with the emergence of the Java language itself
 - They allowed to execute compiled code of the appropriate Java class **on the client side** (browser) with some limitations (in the "sandbox").
 - Since 2016, applets have become "deprecated" and browsers have begun to definitively abandon their support.
- By changing the prefix from "**ap**" to "**serv**", **servlets** were created, i.e. Java classes running **on the server side**.
- They run in a secure application server environment (e.g. GlassFish, last version from 2017) or a web container (e.g. Apache Tomcat, current version 11.0.14 from 2025-11-10). As part of the JEE platform, servlets have access to the entire Java API.
- In addition to the servlet, a server written in Java has a strictly defined architecture (next slide):

Servlet control architecture and flow



Control flow 1/2

- At the beginning, the server application uses the reflection mechanism (including annotation analysis) to browse application classes: controllers, pattern engines, data models, etc. Thanks to this, it prepares the data needed for further request processing (**Handler Mapping**) and rendering page patterns (View Resolver).
 - Creates e.g. controller instances.
- The server (Java application) receives an HTTP request to execute (1). Initial analysis involves parsing and "packing" the data from the request (request line, headers, request body) into the appropriate class object and pre-processing using filters (the mechanism will be explained later). Then the **Front Controller** servlet to pre-handle the request is selected. Default this will be a `DispatcherServlet` object.
- `DispatcherServlet` uses the information collected in **Handler Mapping** to select (2) the appropriate controller that will ultimately handle the request.
- The selected controller prepares the data for creating the page. In many cases, this means reaching (3) to the data model (`DataModel`).
- A data model can contain data in server memory, but **most often** communicates (4a,4b) with an **external database (Database)**
- When it receives data from the data model (5), the controller converts that data into a view model. In practice, communication (3) and (5), and related communication (4a,4b), can take place many times before the controller receives **all the data** needed to create a view model.

Control flow 2/2

- The controller exits by returning (6) in the simplest case the view model (**ViewModel**) and the file name with the view master (**ViewName**).
- `DispatcherServlet` uses the information collected in the View Resolver and the file name (`ViewName`) to select (7) the appropriate **viewing engine**. It provides him with data in **ViewModel** and a specific template file.
- The view creation engine processes the uploaded data to finally produce and return (8) the correct HTML document to `DispatcherServlet`.
- This front-end controller adds response lines, response headers and sends (9) as a **text stream** to the client.

POJO i JavaBeans POJO

- POJO - Plain Old Java Object
 - is a Java object that is not bound by any restrictions other than those enforced by the Java specification. This means that POJO **should not**:
 - Extend from a predetermined class
 - Implement a predetermined interface
 - Contain predetermined annotations
 - This is the **opposite approach** than **previous frameworks** (EJB, Struts, etc.)
- JavaBeans POJO – POJO, but it introduces a strict set of rules on how to implement it:
 - Access levels – properties are private and provide getters and setters
 - Method names – its getters and setters follow the `getX()` and `setX()` conventions (in the case of a `Boolean` value, `isX()` can be used as a getter)
 - Default constructor—An argumentless constructor must be present so that you can create an instance without providing arguments, for example when deserializing
 - Serializable – implementation of the `Serializable` interface allows you to store state
- In short, it will be POJO in the sense of JavaBeans POJO.
- Spring uses such POJOs.

Beans and their injection

- JavaBean is a component in Spring programming (but also in other frameworks), which should be with an appropriate general annotation (`@Bean`) or indicating the role of such a bean, e.g. `@Controller`, `@Service`, etc.
 - **Controllers** have methods (**actions**) with mapping them to the appropriate URL pattern.
 - When the server (application) starts, classes with such an annotation are browsed (through the reflection mechanism) in search of such methods
 - Similarly, the server browses the remaining beans classes
- Beans with the appropriate annotation can be injected into other beans (in the constructor, or into the field by the corresponding field annotation, or as arguments to methods).
- The creation of beans, their injection, etc. is handled by the server in response to the request:
 - Looks for the controller and its action that maps the given request pattern
 - Checks what other beans the controller needs
 - Creates instances of these beans. If more beans are required to create them, it will first create the necessary beans, etc. A beans construction tree is created, where the root is the selected controller.

Extended example: StudentList application - assumptions

- Student collection management app
- **No external database**
 - Collection remembered in memory only for the time of server operation
- Implement all CRUD operations, i.e.:
 - (Create) Creating a new student
 - (Read) Show student data
 - (Update) Edit student data
 - (Delete) Deletion of student data
- The above operations should be in a class that **in the future** can be converted into a class operating on an external database.
- In addition, presentation **of all students** in **tabular form**.
 - After each CRUD operation, automatically return to tabular form to present possible changes.
- To speed up the demonstration, the ability to "seed" the collection with sample data.
- **Data validation will be omitted.**

Steps to create an app

- **Creating** a project (as for a simple example)
 - Dependents: Spring Web, Thymeleaf
- **Creation** of POJO Student
- **Creation** of the `StudentService` **service** (with the `@Service` annotation) that will manage the list of students.
 - Place for all CRUD and ancillary operations for student collections
 - `import org.springframework.stereotype.Service;`
- **Creation** of a `StudentController` (with an annotation `@Controller`) that will manage request handling (based on URL path):
 - `/student/` - (GET) Student table
 - `/student/seed` – (GET) seeding the collection (adding students during application launch)
 - `/student/add` – (GET) adding a student, presenting a form to fill in with empty fields
 - `/student/add` – (POST) adding a student to a collection, student data in the POST request payload
 - `/student/details?id=3` – (GET) showing student data with `id=3`
 - `/student/3/edit` – (GET) preparation of a form for changing student data, filled with student data with `id=3`
 - Especially a slightly different URL pattern, to present this type of pattern, `/student/edit?id=3` would be better
 - `/student/edit` – (POST) change of student data, data in the request payload (there is also a student `id`)
 - `/student/remove?id=3` – delete a student with `id=3`
- Preparation of **page templates** for the above-mentioned action

POJO Student

- Student class with private fields and getters and setters to each of them
 - In most environments there is an option to **generate getters and setters** automatically, in IntelliJ it is the <Alt>+<Ins> shortcut and select the appropriate option
- There is a **default constructor** (required by Spring Boot)
- There is a second constructor (facilitates management in the student collection)
 - Also created by keyboard shortcut

```
package com.example.studentlist;

public class Student {
    private long id;
    private String firstName;
    private String lastName;
    private int index;
    private String login;
    private String password;

    public Student() {
    }

    public Student(long id, String firstName, String lastName, int index, String login, String password) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.index = index;
        this.login = login;
        this.password = password;
    }

    public long getId() {
        return id;
    }
    public void setId(long id) {
        this.id = id;
    }
    // rest of getters and setters
}
```

Service StudentService 1/2

- For simplicity, use inside the `ArrayList<>` class
- This service does not need anything to work, so there is only a non-parametric constructor.
- After adding an annotation `@Service` the IDE itself adds the appropriate package, i.e. it added:
 - `import org.springframework.stereotype.Service;`
- Similarly, when using a class that is not imported, the IDE proposes to import the corresponding class using a combination `<Alt>+<Shift>+<Enter>` (or sometimes `<Alt>+<Enter>`)
- *Imperfection:* Adding a student does not check if such an `id` already exists

```
package com.example.studentlist;

import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.List;

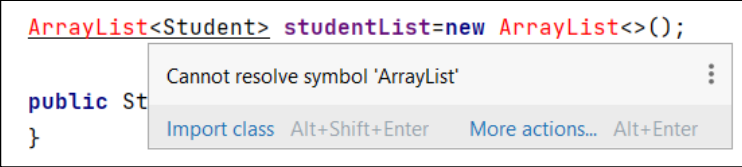
@Service
public class StudentService {
    ArrayList<Student> studentList=new ArrayList<>();

    public StudentService() {
    }
    public void seed(){
        studentList.add(new Student(1,"Adam","Fox",12345,"stud1","abc"));
        studentList.add(new Student(2,"Ewa","Snake",12346,"stud2","aaa"));
        studentList.add(new Student(3,"John","Brown",22222,"stud3","eightstars"));
    }

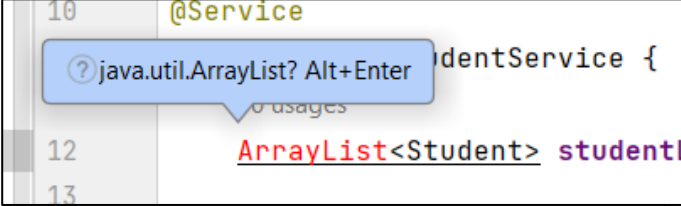
    private boolean isEmpty() {
        return studentList.size() == 0;
    }

    public List<Student> getAllStudent() {
        return studentList;
    }

    public void addStudent(Student student) {
        studentList.add(student);
    }
}
```



The screenshot shows a code completion popup for the `ArrayList` class. The popup text is: "Cannot resolve symbol 'ArrayList'". Below this, there are two options: "Import class Alt+Shift+Enter" and "More actions... Alt+Enter".



The screenshot shows a code completion popup for the `java.util.ArrayList` class. The popup text is: "? java.util.ArrayList? Alt+Enter". Below this, there are two options: "Usages" and "ArrayList<Student> studentList".

Service StudentService 2/2

- *Imperfection:* The student update method inserts it at the end of the list (it removes the previous one earlier)
- *Imperfection:* Removing a student does not check whether the student actually exists.
- The imperfections listed above will be used to present various behaviors of the resulting application.
 - In practice, they should be improved

```
public Student getStudentById(long id) {  
    for(Student student:studentList){  
        if(student.getId()==id)  
            return student;  
    }  
    return null;  
}  
  
public Student getStudent(Student student){  
    return getStudentById(student.getId());  
}  
  
public void updateStudent(Student student) {  
    deleteStudent(student);  
    studentList.add(student);  
}  
  
public void deleteStudent(Student student) {  
    studentList.remove(getStudentById(student.getId()));  
}  
  
public void deleteStudentById(long id) {  
    studentList.remove(getStudentById(id));  
}  
}
```

Controller StudentController - begin

- Add annotation `@Controller`
- Package imports performed by the environment
 - `org.springframework.stereotype.Controller`
- This time the controller **requires** `StudentService` to work, hence its **injection** in the constructor
- Of course, it has to be stored somewhere, hence the `studentService` field in this class.
- The **following slides** show the **controller's methods** and fragments of the table view responsible for calling the method.

```
package com.example.studentlist;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.text.DateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

@Controller
public class StudentController {
    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }
}
```

StudentController – Show all students

- The annotation `@GetMapping("/student/")` before the `home()` method means that if a GET request is sent with the path `"/student/"`, this method is to be run.
- Inject a model in a method call
 - This is the model for the view pattern
 - It can also be done similarly, without injection, but injecting is the simplest.
- In order to **demonstrate** under the table of students to appear the **current server time**
 - For a clear presentation of the time, you can **inject** the `Locale` service, although we have not formally marked it as a service (there are many such classes that can be injected in methods)
- The following are inserted into the model: „`serverTime`” (as `String`) and „`studentList`” (as `List<Student>`) using appropriate services.
- We return the name of the view `student/index`, which is in the folder `resources/templates/student`

```
@GetMapping("/student/")
public String home(Locale locale, Model model) {
    Date date = new Date();
    DateFormat dateFormat = DateFormat.getDateTimeInstance(DateFormat.LONG,
        DateFormat.LONG, locale);
    String serverTime = dateFormat.format(date);
    model.addAttribute("serverTime", serverTime.toString());
    List<Student> studentList = studentService.getAllStudent();
    model.addAttribute("studentList", studentList);
    return "student/index";
}
```

View template student/index.html 1/2

- To use the capabilities of the Thymeleaf view engine, you must add a line:
 - `<html xmlns:th="http://www.thymeleaf.org en">`
- This means that attributes starting with "th:" will be specially processed.
- For **easier styling**, the **Bootstrap** library (<https://getbootstrap.com/>) will be used. In this case, it is usually enough to add only the appropriate class to the HTML element. To use this library, just add one below line, starting with `<link rel="stylesheet" ...>`.
 - An example of use is `class="btn btn-primary"`
- The last two lines of this part of the code are buttons to call data seeding and add a student.

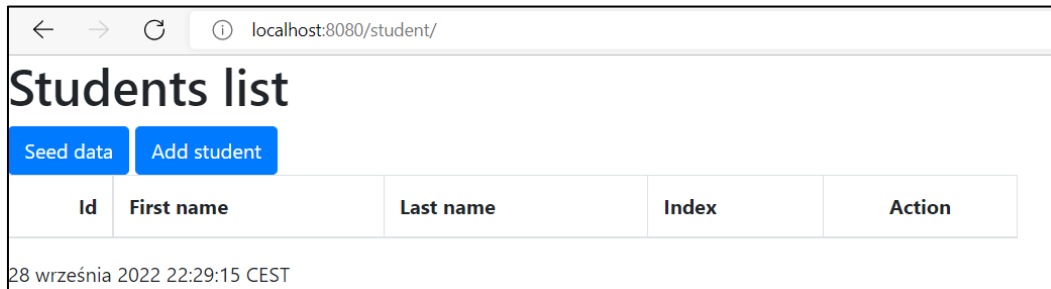
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
  <meta charset="text/html; UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <title>Students list</title>
</head>
<body>
<h1>Students list</h1>
<a class="btn btn-primary" href="/student/seed">Seed data</a>
<a class="btn btn-primary" href="/student/add">Add student</a>
```

View template student/index.html 2/2

```
<div class="content">
  <table class="display table table-bordered" style="width:800px">
    <thead class="text-center">
      <tr>
        <th class="text-right">Id</th>
        <th class="text-left">First name</th>
        <th class="text-left">Last name</th>
        <th class="text-left">Index</th>
        <th class="text-center" colspan="3">Action</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="student : ${studentList}">
        <td class="text-right" th:text= "${student.getId()}"></td>
        <td class="text-left" th:text= "${student.getFirstName()}"></td>
        <td class="text-left" th:text= "${student.getLastName()}"></td>
        <td class="text-center" th:text= "${student.getIndex()}"></td>
        <td><a class="btn btn-primary" th:href="@{/student/details(id=${student.id})}">details</a></td>
        <td><a class="btn btn-primary" th:href="@{/student/{studentId}/edit(studentId=${student.getId()})}">edit</a></td>
        <td><a class="btn btn-danger" th:href="@{/student/remove(id=${student.id})}">remove</a></td>
      </tr>
    </tbody>
  </table>
</div>
<p th:text="${serverTime}"></p>
</body>
</html>
```

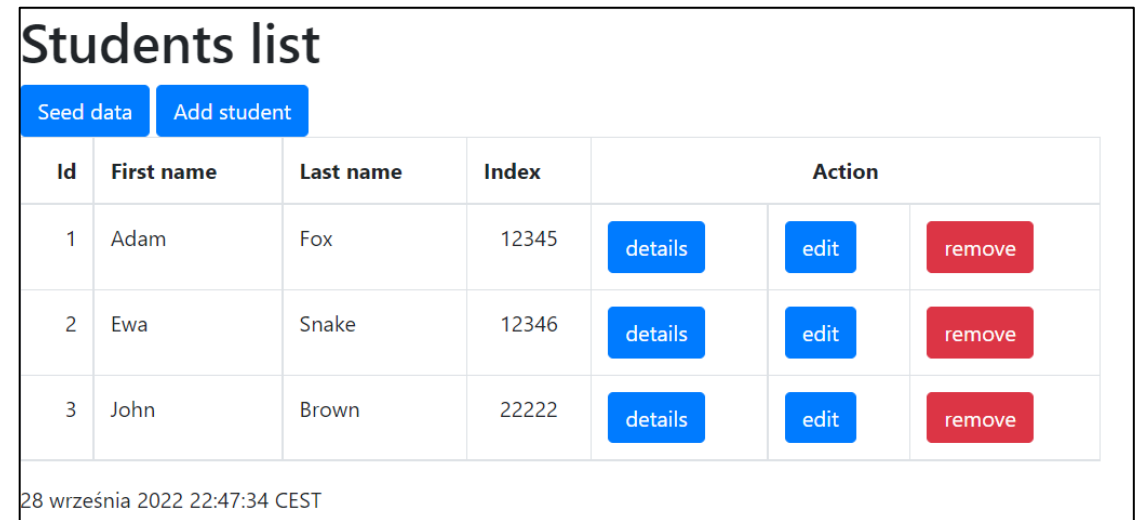
View template student/index.html - description

- Elements (buttons) of the type `<a ...="">` **in the table** will be described elsewhere
- Element `<tr th:each="student : ${studentList}">` means that many more `<tr>` elements will be produced, each for the `student` object that will be taken from the `studentList` collection one by one. Text inside `<tr>` `</tr>` will also be created for each of these objects.
- Element `<td class="text-right" th:text= "${student.getId()}"></td>` uses the above-mentioned `student` object and runs `getId()` for it and inserts **as content** (attribute `th:text`) of the element `<td>` (even though it looks like the interior is empty)
- Similarly, the other student fields are run.
- Below the table inserted server time using a similar command for the paragraph:
 - `<p th:text="${serverTime}"></p>`
- Below is the **initial** appearance and after **sowing** the elements:



Id	First name	Last name	Index	Action
----	------------	-----------	-------	--------

28 września 2022 22:29:15 CEST



Id	First name	Last name	Index	Action
1	Adam	Fox	12345	details edit remove
2	Ewa	Snake	12346	details edit remove
3	John	Brown	22222	details edit remove

28 września 2022 22:47:34 CEST

Page student/index.html – page source

- In the browser, it is worth seeing what HTML document was created:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="text/html; UTF-8">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.r
sRII4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkulAvNWLzeN8tE5YBujZqJLB" cros
/dAiS6JXm" crossorigin="anonymous">
  <title>Students list</title>
</head>
<body>
<h1>Students list</h1>
<a class="btn btn-primary" href="/student/seed">Seed data</a>
<a class="btn btn-primary" href="/student/add">Add student</a>
<div class="content">
  <table class="display table table-bordered" style="width:800px">
    <thead class="text-center">
      <tr>
        <th class="text-right">Id</th>
        <th class="text-left">First name</th>
        <th class="text-left">Last name</th>
        <th class="text-left">Index</th>
        <th class="text-center" colspan="3">Action</th>
      </tr>
    </thead>
```

```
<tbody>
  <tr>
    <td class="text-right">1</td>
    <td class="text-left">Adam</td>
    <td class="text-left">Fox</td>
    <td class="text-center">12345</td>
    <td><a class="btn btn-primary" href="/student/details?id=1">details</a></td>
    <td><a class="btn btn-primary" href="/student/1/edit">edit</a></td>
    <td><a class="btn btn-danger" href="/student/remove?id=1">remove</a></td>
  </tr>
  ...
  <td class="text-right">3</td>
  <td class="text-left">John</td>
  <td class="text-left">Brown</td>
  <td class="text-center">22222</td>
  <td><a class="btn btn-primary" href="/student/details?id=3">details</a></td>
  <td><a class="btn btn-primary" href="/student/3/edit">edit</a></td>
  <td><a class="btn btn-danger" href="/student/remove?id=3">remove</a></td>
</tr>
</tbody>
</table>
</div>
<p>28 września 2022 23:05:25 CEST</p>
</body>
</html>
```

Data seeding

```
@GetMapping("/student/seed")  
public String seed() {  
    studentService.seed();  
    return "redirect:/student/";  
}
```

```
<a class="btn btn-primary" href="/student/seed">Seed data</a>
```

- Above the method from the controller.
- Below is a fragment from the table view pattern.
- For a GET request with the path ("/student/seed"), the seed() method is called.
 - And such a path is for a button in a tabular view.
- The path to the page pattern is a string that begins with "redirect:", which means that the browser should, after receiving the response, perform **another GET request** to the address after this string, in this case "/student/".
 - which will call home (...) action.
- If only "/student/" were used, the view engine **would not receive** data for the view (or you would have to execute the same code as for the home() method)).

Add a student

```
@GetMapping("/student/add")
public String add(Model model) {
    model.addAttribute("student", new Student() );
    return "student/add";
}

@PostMapping("/student/add")
public String add(@ModelAttribute Student student) {
    studentService.addStudent(student);
    return "redirect:/student/";
}
```

```
<a class="btn btn-primary" href="/student/add">Add student</a>
```

- Both of the above methods have as a path pattern `/student/add`
- The **first method** runs for the GET request, prepares the "empty" `Student` class object and sends it to the `student/add` view (with form, next slide)
- The second method runs for the POST request. An annotation `@ModelAttribute` **before** the `Student` class **parameter** means that the data **from the payload** of this request will be rewritten **field by field** (using **setters**) to the `Student` class object, and the prepared object will be **injected** as a `student` parameter to call this method.
- Similarly to seeding, you are redirected to a page with a table.

View template student/add.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
  <meta charset="text/html; UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        integrity="sha384-sRl14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <title>Add student form</title>
</head>
<body>
<h1>Add a new student</h1>
<form action="#" th:action="@{/student/add}" th:object="${student}" method="post">
  <p>Id: <input type="text" th:field="*{id}" /></p>
  <p>First name: <input type="text" th:field="*{firstName}" /></p>
  <p>Last name: <input type="text" th:field="*{lastName}" /></p>
  <p>index: <input type="number" th:field="*{index}" /></p>
  <p>login: <input type="text" th:field="*{login}" /></p>
  <p>password: <input type="text" th:field="*{password}" /></p>
  <p><input type="submit" value="Add" /> <input type="reset" value="Reset" /></p>
</form></body></html>
```

- Attribute `th:action="@{/student/add}"` converts the text in the action attribute to `/student/add` (@ is the relative path of the application, in this case it is empty)
- Attribute `th:object="${student}"` inside the element `<form>` allows you to use shorter notation. Instead of e.g. `th:field="${student.firstName}"` you can shorter: `th:field="*{firstName}"`
- Attribute `th:field` performs two operations:
 - Inserts the value of this object field into the item `<input>` (using the field getter)
 - When you create a form, sets the `name` attribute to the name of the object field (which allows the annotation `@ModelAttribute` to work)

Adding a student – the effect of the action

localhost:8080/student/add

Add a new student

Id:

First name:

Last name:

Index:

Login:

Password:

localhost:8080/student/add

Add a new student

Id:

First name:

Last name:

Index:

Login:

Password:

Students list

Id	First name	Last name	Index	Action		
1	Adam	Fox	12345	details	edit	remove
2	Ewa	Snake	12346	details	edit	remove
3	John	Brown	22222	details	edit	remove
5	Dariusz	Konieczny	22222	details	edit	remove

28 września 2022 23:41:01 CEST

- TODO – If there is time during the lecture - check the source of the page for adding a student.

Remove a student

```
// how to put a parameter in a query string
// e.a. /student/remove?id=3
@GetMapping("/student/remove")
public String remove(@RequestParam("id") long id) {
    studentService.deleteStudentById(id);
    return "redirect:/student/";
}
```

```
<td><a class="btn btn-danger" th:href="@{/student/remove(id=${student.id})}">remove</a></td>
```

- The `remove()` method runs on a URL of type `/student/remove?id=3`. This is indicated by two annotations:
 - `@GetMapping("/student/remove")` – Before the method
 - `@RequestParam("id")` – before its `long id` parameter to be mapped with the `id` parameter from the query string. Before the server runs this method, it parses the query string `"?id=3"` and creates a parameter of the method from the `id` value (here the name of the method parameter is only accidentally the same as the name of the query string parameter).
- In the page pattern, `th:href="@{/student/remove(id=${student.id})}"` has a set of local "variables" between the parentheses (in the example one variable `id`) and their assigned values. If the variable is **not used before the parenthesis**, it will be sent as a **query string** as a pair `"name=value"`.
- After removing a student on the site, redirect to a page with a table.

Student Edit

```
// how to put parameter in an URL
// e.a. /student/3/edit
@GetMapping(value = {"/student/{studId}/edit"})
public String edit(Model model, @PathVariable Integer studId) {
    model.addAttribute("student", studentService.getStudentById(studId) );
    return "/student/edit";
}

@PostMapping(value = {"/student/edit"})
public String edit(@ModelAttribute Student student) {
    studentService.updateStudent(student);
    return "redirect:/student/";
}
```

```
<td><a class="btn btn-primary" th:href="@{/student/{studentId}/edit(studentId=${student.getId()})}">edit</a></td>
```

- Student editing (like adding) is a two-step process
 - Creating a page with a form filled in with student data
 - After approval – student update and return to the table page.
- Demonstration of how to place data in a URL path, e.g. /student/3/edit
- In the controller:
 - @GetMapping(value = {"/student/{studId}/edit"}) **before the method**
 - @PathVariable Integer studId – **before the method parameter**. This time the names must be identical (studId).

Another way to place parameters on a URL path

```
<td><a class="btn btn-primary" th:href="@{/student/{studentId}/edit(studentId=${student.getId()})}">edit</a></td>
```

- In the table view template:
 - `th:href="@{/student/{studentId}/edit(studentId=${student.getId()})}"`
 - This time, the local variable (in round brackets) `studentId` **is used** in the path pattern `/student/{studentId}/edit`, so its value will be inserted there (and **not into the query string**). Of the variables in parentheses, when only a part is used on a URL path, **the rest** is passed as a **query string**.
- The `edit()` method for POST is similar in action as `add()`

View student/edit.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
  <meta charset="text/html; UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <title>Edit student form</title>
</head>
<body>
<h1>Edit student</h1>
<form action="#" th:action="@{/student/edit}" th:object="${student}" method="post">
  <p>Id: <input type="text" th:field="*{id}" /></p>
  <p>First name: <input type="text" th:field="*{firstName}" /></p>
  <p>Last name: <input type="text" th:field="*{lastName}" /></p>
  <p>index: <input type="number" th:field="*{index}" /></p>
  <p>login: <input type="text" th:field="*{login}" /></p>
  <p>password: <input type="text" th:field="*{password}" /></p>
  <p><input type="submit" value="Modify" /> <input type="reset" value="Reset" /></p>
</form>
</body>
</html>
```

- A view very similar, and works similar to a view student/add.html. The differences are <title>, <h1>, th:action and value=„Modify” for the button <input type="submit" ...>
- Notation, e.g. th:field="*{firstName}" causes student data to become the initial content of form fields (which affects the behavior of the button <input type="reset" ...="">
- Data from the form is sent to /student/edit in the form of a POST request.

Preview student data, view student/details.html

```
@GetMapping("/student/details")
public String add(@RequestParam("id") long inputId, Model model) {
    model.addAttribute("student", studentService.getStudentById(inputId) );
    return "/student/details";
}
```

```
<td><a class="btn btn-primary" th:href="@{/student/details(id=${student.id})}">details</a></td>
```

- Annotations in the method and code in table view with the mechanisms previously discussed.
- For details view, use `th:object="${student}"` for an item `<table>` to use EL with an asterisk in table fields (for example, `th:text="*{firstName}"`)

```
<title>Details</title>
</head>
<body>
<h1>Details</h1>
<p>
<table class="table table-striped table-bordered" th:object="${student}">
<thead>
<tr>
<th>Property</th>
<th>Value</th>
</tr>
</thead>
<tbody>
<tr>
<td>Id</td>
<td th:text="*{id}"></td>
</tr>
<tr>
<td>First name</td>
<td th:text="*{firstName}"></td>
</tr>
```

```
<tr>
<td>Last name</td>
<td th:text="*{lastName}"></td>
</tr>
<tr>
<td>Index</td>
<td th:text="*{index}"></td>
</tr>
<tr>
<td>Login</td>
<td th:text="*{login}"></td>
</tr>
<tr>
<td>Password</td>
<td class="table-danger" th:text="*{password}"></td>
</tr>
</tbody>
</table>
</p>
</body>
</html>
```


The effect of the methods of presenting student data

Students list

Seed data Add student

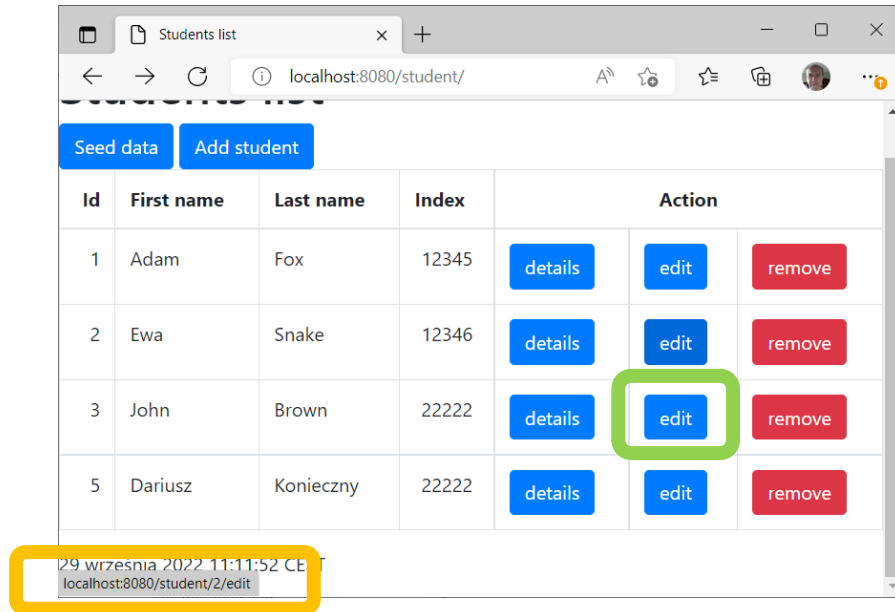
Id	First name	Last name	Index	Action
1	Adam	Fox	12345	details edit remove
2	Ewa	Snake	12346	details edit remove
3	John	Brown	22222	details edit remove
5	Dariusz	Konieczny	22222	details edit remove

localhost:8080/student/details?id=2

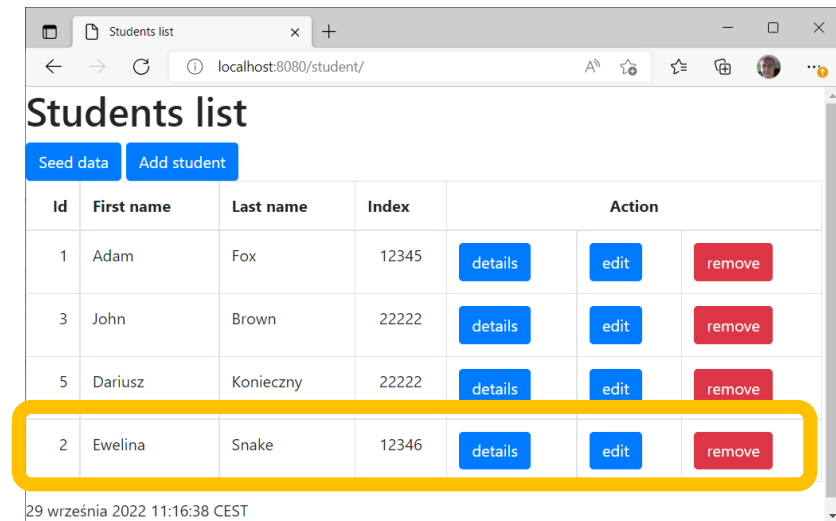
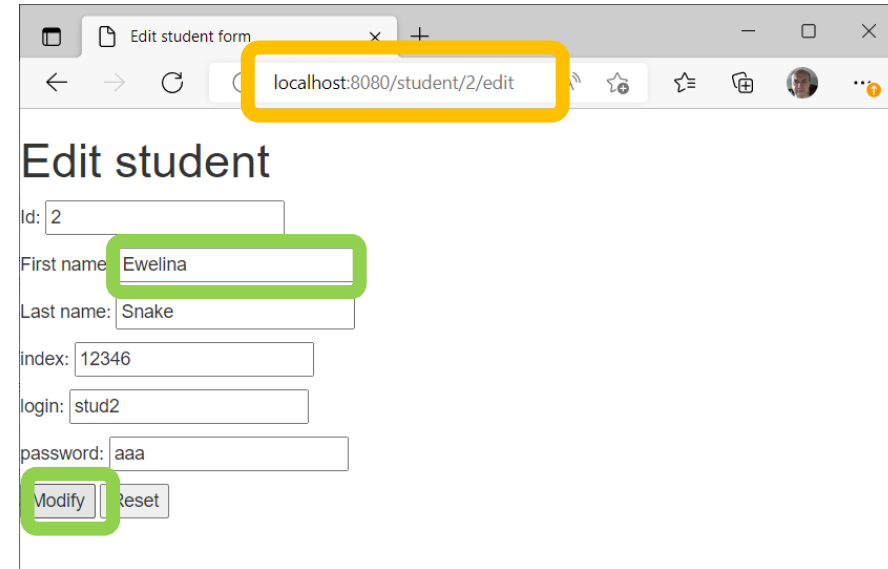
Details

Property	Value
Id	2
First name	Ewa
Last name	Snake
Index	12346
Login	stud2
Password	aaa

Presentation of student editing activities



- After modification, the student is **at the end of the list**



Demonstration of the action of deleting a student

Students list

Seed data Add student

Id	First name	Last name	Index	Action		
1	Adam	Fox	12345	details	edit	remove
3	John	Brown	22222	details	edit	remove
5	Dariusz	Konieczny	22222	details	edit	remove
2	Ewelina	Snake	12346	details	edit	remove

localhost:8080/student/remove?id=5

Students list

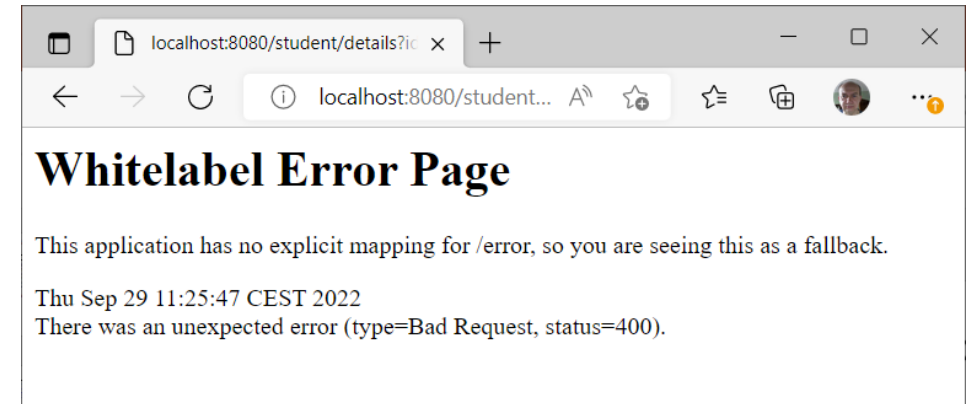
Seed data Add student

Id	First name	Last name	Index	Action		
1	Adam	Fox	12345	details	edit	remove
3	John	Brown	22222	details	edit	remove
2	Ewelina	Snake	12346	details	edit	remove

29 września 2022 11:20:22 CEST

Notes for execution

- How the program responds to an incorrect URL
 - `/student/what`
- How it responds to incorrect parameters in the URL
 - `/student/details?id=50`
 - `/student/50/edit`
 - `/student/details?id=xyz`
- In most cases, the error page.
- Forms should be supplemented with HTML codes for data validation.
- But this does not protect against incorrect data that reaches the server, because you can use tools such as `curl`, `postman`, `proxy` to modify the POST query.
- Server-side validation must be re-performed.
 - As well as, for example, check if the `id` is already in the database or even create an `id` only on the server side.



Start the server

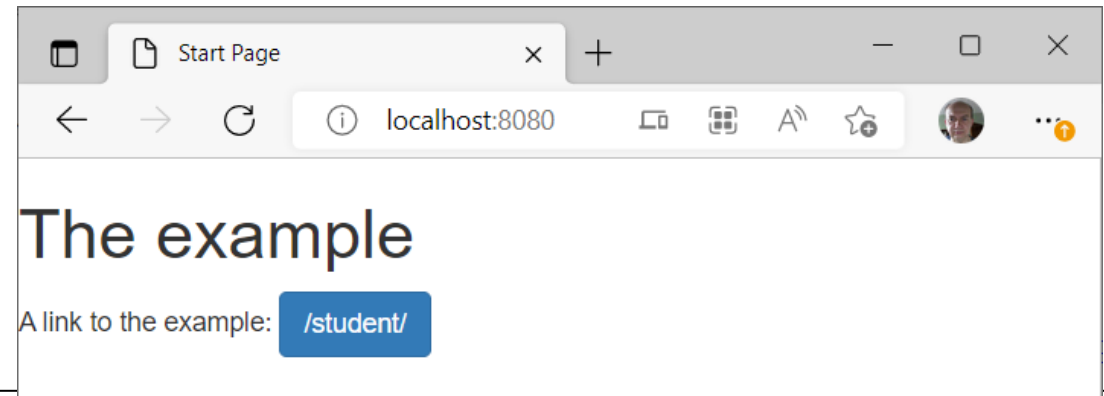
```
c.e.studentlist.StudentListApplication : Starting StudentListApplication using Java 17.0.2 on DK-Dell with PID 43184
(C:\Users\dariu\IdeaProjects\StudentList\target\classes started by dariu in C:\Users\dariu\IdeaProjects\StudentList)
c.e.studentlist.StudentListApplication : No active profile set, falling back to 1 default profile: "default"
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
o.apache.catalina.core.StandardService : Starting service [Tomcat]
org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.65]
o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 572 ms
o.s.b.a.w.s.WelcomePageHandlerMapping : Adding welcome page template: index
o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
c.e.studentlist.StudentListApplication : Started StudentListApplication in 1.077 seconds (JVM running for 1.457)
```

- The server starts the class `StudentListApplication`
- Sets the default profile
- Initializes Tomcat (embedded) on port 8080 (and eventually starts it)
- Sets data from the context of the application
- Adds the home page as `index`
 - This means that a default routing rule is created for `'\'`, which will default to the `index` view (for Thymeleaf it will be `index.html`)

View `/index.html`

- Let's create such a page in the `templates` folder (code below).
- It really works!
- This is the **only** routing rule, creating another view file in the `templates` folder (or subfolders, such as `/a/probe.html`) does not automatically access it.
- The `/index` or `/index.htm` URLs also **don't work**.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
  <meta charset="text/html; UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <title>Start Page</title>
</head>
<body>
<h1>The example</h1>
<p>
  A link to the example:
  <a class="btn btn-primary" href="/student/"> /student/ </a>
</p>
</body>
</html>
```

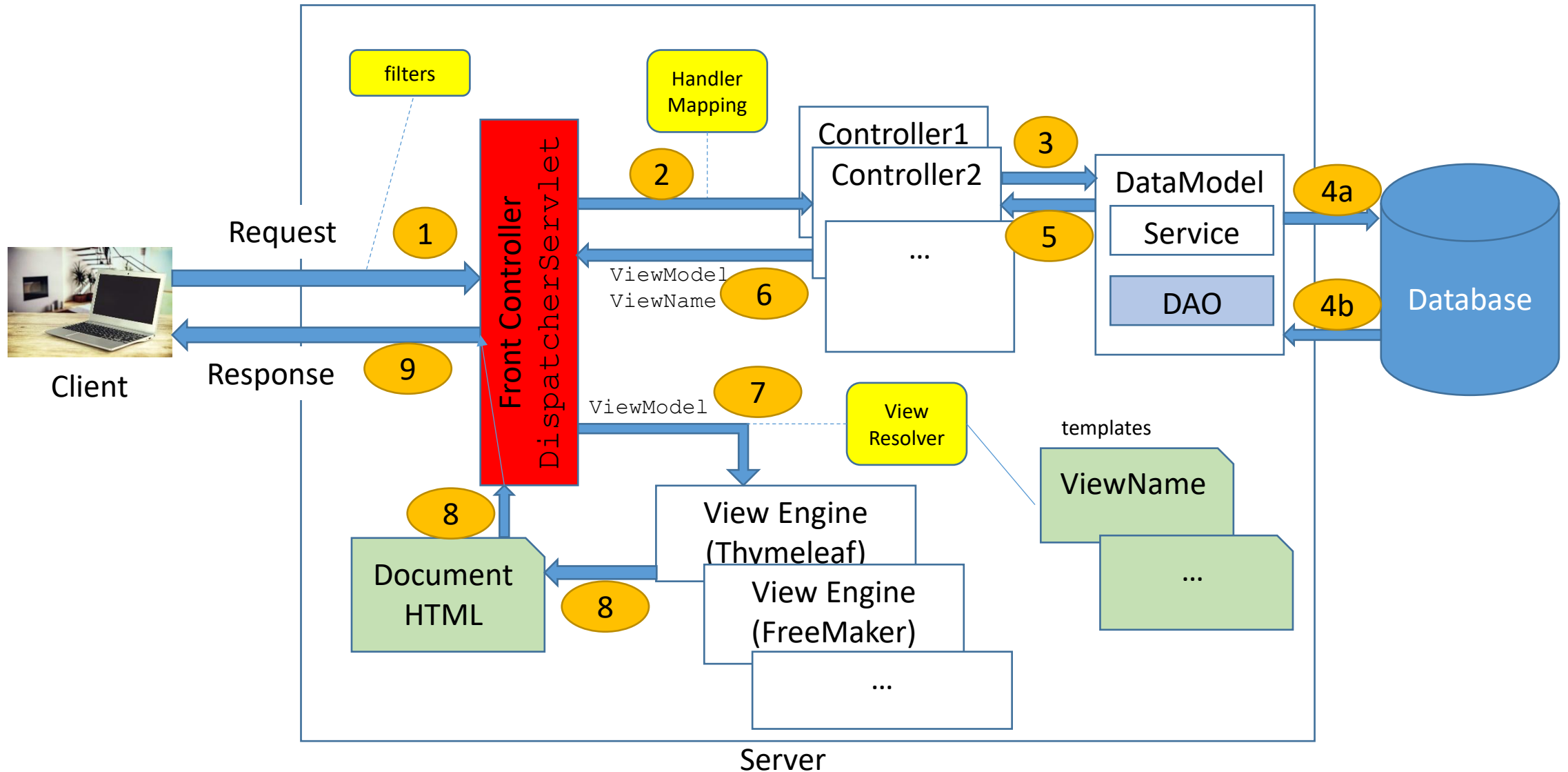


Continuing Server Execution

- The server works in a **lazy way**, only the **first request** creates an object of the `DispatcherServlet` class (this can be observed in the console).
- In the console, you can also observe, for example, various errors (e.g. incorrect spelling with "th:" or error parsing the URL of the request).
- Example (below, from the console) when you launch the app:
 - URL: /
 - URL: /student/**xyz**/edit/
- **TODO: How StudentList works in the diagram on the next page**

```
o.a.c.c.C.[Tomcat].[localhost].[/]      : Initializing Spring DispatcherServlet 'dispatcherServlet'
o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
.w.s.m.s.DefaultHandlerExceptionResolver : Resolved
[org.springframework.web.method.annotation.MethodArgumentTypeMismatchException: Failed to convert value of type
'java.lang.String' to required type 'java.lang.Integer'; nested exception is java.lang.NumberFormatException: For
input string: "xyz"]
```

Servlet control architecture and flow



Fragments in Thymeleaf

Additional material 1/2 – if time allows it during the lecture, if not – for your own study.

Fragments in Thymeleaf

- In the basic Thymeleaf package, there is a fragment mechanism - **Thymeleaf Standard Layout System**.
- Lets you insert HTML code fragments into other html. Thus, repetitive parts of html code can be separated into separate files and inserted into the code with the appropriate Thymeleaf attribute in a selected place (as if calling a method in programming)
 - It is called **include-style layouts**
- Example - modifying a project with a list of students to have a menu with links to: page with a table, data seeding operations, adding a new student. In addition, using a different fragment, adding a common footer for all pages.
 - Of course, the top buttons in the student table view will be removed.
 - Bootstrap will be used along with Javascript codes (in the <head>).
- `Project StudentListFragments`.
 - Dependencies: Spring Web, Thymeleaf
 - The fragments will be placed in the folder `templates/fragment/`

Modifications to the view student/index.html

- **Similar** changes should be made in the view student/add.html, student/details.html, student/edit.html
- Commented out elements will be discussed in later slides.
- This time we need also **scripts** written for **Bootstrap**.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
  <meta charset="text/html: UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous">
  <title>Students list</title>
</head>
<body>
  <header th:replace="~{fragment/header::myheader}"> </header>
  <!-- <header th:replace="~{fragment/layout::myheader}"> </header> -->
  <main>
    <h1>Students list</h1>
    <div class="content">
      ...
    </div>
    <p th:text="${serverTime}"></p>
  </main>
  <footer th:replace="~{fragment/footer::myfooter}"> </footer>
  <!-- <footer th:replace="~{fragment/layout::myfooter}"> </footer> -->
</body>
</html>
```

Use fragments

- In the element `<header>`, attribute `th:replace="~{fragment/header::myheader}"` indicates that the fragment named `myheader` (after the double colon) from the `fragment/header` view pattern (before the double colon) is used here. That is, from the folder `fragment` the `header.html` file.
 - The `~{}` means "**link expression**" (from Thymeleaf 3)
- Similarly, in the element `<footer>` attribute `th:replace="~{fragment/footer::myfooter}"` means the use of a fragment `myfooter` from file `fragment/footer.html`.
- An element `<main>` has also been added to the code that contains all previous page content.
 - which allows you to see that all page patterns differ basically only in the element `<main>`.

Fragments codes

- Fragments codes: `fragment/header.html`, `fragment/footer.html`
- Each fragment has a `th:fragment` attribute with its name (`myheader` or `myfooter`)

```
<header th:fragment="myheader">
  <nav class="navbar navbar-dark bg-primary">
    <div class="container">
      <!-- Logo + menu in one flex container -->
      <div class="d-flex align-items-center">
        <a class="navbar-brand" href="/student/">StudentList</a>
        <ul class="navbar-nav flex-row ms-3"> <!-- ms-3 = margin-left -->
          <li class="nav-item">
            <a class="nav-link px-2" href="/student/">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link px-2" href="/student/seed">Seed</a>
          </li>
          <li class="nav-item">
            <a class="nav-link px-2" href="/student/add">Add new</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
</header>
```

```
<footer th:fragment="myfooter">
  <p>
    &copy;Dariusz Konieczny
  </p>
</footer>
```

Effect of fragments work

- The top menu and footer **appear the same** on all pages.

[StudentList](#) [Home](#) [Seed](#) [Add new](#)

Students list

Id	First name	Last name	Index	Action		
1	Adam	Fox	12345	details	edit	remove
2	Ewa	Snake	12346	details	edit	remove
3	John	Brown	22222	details	edit	remove

25 października 2022 11:11:07 CEST

©Dariusz Konieczny

[StudentList](#) [Home](#) [Seed](#) [Add new](#)

Add a new student

Id:

First name:

Last name:

index:

login:

password:

©Dariusz Konieczny

Fragments in one file

- Because fragments are determined by their name in the `th:fragment` attribute, they can be in a single file (for example `fragment/layout.html`)
 - To test it, uncomment the corresponding lines in the view files (and comment out the previously used ones).
- Fragments can be placed inside code that creates **valid HTML** page code
 - Then you can see these pages without running the app

```
<header th:fragment="myheader">
  <nav class="navbar navbar-dark bg-primary">
    <div class="container">
      <!-- Logo + menu in one flex container -->
      <div class="d-flex align-items-center">
        <a class="navbar-brand" href="/student/">StudentList</a>
        <ul class="navbar-nav flex-row ms-3"> <!-- ms-3 = margin-left -->
          <li class="nav-item">
            <a class="nav-link px-2" href="/student/">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link px-2" href="/student/seed">Seed</a>
          </li>
          <li class="nav-item">
            <a class="nav-link px-2" href="/student/add">Add new</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
</header>
<!-- This will be not used -->
<p>
  Not used part.
</p>
<footer th:fragment="myfooter">
  <p>
    &copy;Dariusz Konieczny
  </p>
</footer>
```

Summary of fragments in Thymeleaf

- Advantages
 - Easy to use and understand.
 - Minimalist: you can only write code for common fragments (not the whole page)
 - When you add the fragment code inside the correct HTML code for the entire page, you can run it independently in the browser.
 - Flexible: Fragments can be saved one in separate files or more (all) in one file.
- Disadvantages:
 - In each view file:
 - You must separately in each file type the placement of the fragment using `th:replace`
 - You need to enter all the data you need in the item `<head>` and create the code needed for a valid HTML page.
- Solution of defects - use fragments with parameters to prepare layout
 - or use **Thymeleaf Layout Dialect**
 - Advantage - we build the actual layout of the page (layout) to which we insert all or fragments of page patterns (template) - **reversal of dependencies**.

Layouts (by fragments) in Thymeleaf

Additional material 2/2 – if time allows it during the lecture, if not – for your own study.

Layouts in Thymeleaf

- It allows you to create an actual page layout (layout), and then in the page view enter only the fragments needed to fill in the layout.
- `Project StudentListLayout`.
 - Dependencies: Spring Web, Thymeleaf
 - The layout and fragments will be placed in the folder `templates/_shared`
- You should prepare (e.g. in `templates/_shared`) a page layout file
 - Previous standard fragments will also be used: `_header.html` and `_footer.html`, to use code written for a previous project
 - Layout file: `_layout.html`

File _layout.html

- Fragment id with parameters `th:fragment="layout(title, main)"`:
 - `th`: for standard Thymeleaf namespace
- When used the value of parameter `main` it will be inserted in the place `th:replace="${main}"`.
- The same with the parameter `title`.

```
<!DOCTYPE html>
<html lang="en"
      th:fragment="layout(title, main)"
      xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="text/html; UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
          integrity="sha384-EKvoFForCGlvvwx9Hi091cYn3ny7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous">
  <title th:replace="${title}">Students</title>
</head>
<body>
<header th:replace="~{ shared/ header::mvheader}"> </header>
<main th:replace="${main}">
  <p>This content must not be visible.</p>
</main>
<footer th:replace="~{ _shared/ _footer::myfooter}"> </footer>
</body>
</html>
```

View file add.html

- In the view file, (in addition to setting the namespace as in the layout file) use in the element `<head>` pseudoattribute `th:replace` by specifying the path to the file with layout and its parameters.
 - here: HTML elements `<title>` and `<main>`
 - you can name **any element** using pseudoattribute `th:fragment="name"` and then use it as a link expression `~{::name}`.
- Similar changes must be made to the other view files.

```
<!DOCTYPE html>
<html lang="en"
      xmlns:th="http://www.thymeleaf.org"
      th:replace="~{_shared/_layout :: layout(~{::title}, ~{::main})}">

  <title>Add student</title>

  <main>
    <h1>Add a new student</h1>
    <form action="#" th:action="@{/student/add}" th:object="${student}" method="post">
      <p>Id: <input type="text" th:field="*{id}" /></p>
      <p>First name: <input type="text" th:field="*{firstName}" /></p>
      <p>Last name: <input type="text" th:field="*{lastName}" /></p>
      <p>index: <input type="number" th:field="*{index}" /></p>
      <p>login: <input type="text" th:field="*{login}" /></p>
      <p>password: <input type="text" th:field="*{password}" /></p>
      <p><input type="submit" value="Add" /> <input type="reset" value="Reset" /></p>
    </form>
  </main>

</html>
```

How the page layout works

- Creating a view starts with a `student/add.html` file (the original file)
- When encountering `th:replace=~{_shared/_layout :: layout(~{::title}, ~{::main})}`, the Thymeleaf engine prepares values for the arguments from the current file and then replace the current `<html>` element with the one from `_shared/_layout.html` file (layout file).
- When it hits there an element with a pseudo-attribute `th:replace` replaces it with the value of an appropriate argument (e.a. `<title th:replace="${title}">`).
- In the layout file there may be many such places to fill in.
- If the original file **does not contain** a fragment corresponding to the name in the argument, it will generate an exception.
- Other elements of the initial view pattern, which are not parts of the layout, will be ignored when creating the view.
 - However, they can be added so that the whole creates the correct HTML page, if we want to refresh it, for example, on a regular basis in the Live Server without the entire layout, in a simplified version.

Another view file add.html

- An example how to use any element as parameter (id "cos").
 - the <head> element in the layout will be replaced with the following <div> fragment

```
<!DOCTYPE html>
<html lang="en"
  xmlns:th="http://www.thymeleaf.org"
  th:replace="~{_shared/_layout :: layout(~{::title}, ~{::cos})}">
  <title>Add student</title>

  <div th:fragment="cos" >
    <h1>Add a new student</h1>
    <form action="#" th:action="@{/student/add}" th:object="${student}" method="post">
      <p>Id: <input type="text" th:field="*{id}" /></p>
      <p>First name: <input type="text" th:field="*{firstName}" /></p>
      <p>Last name: <input type="text" th:field="*{lastName}" /></p>
      <p>index: <input type="number" th:field="*{index}" /></p>
      <p>login: <input type="text" th:field="*{login}" /></p>
      <p>password: <input type="text" th:field="*{password}" /></p>
      <p><input type="submit" value="Add" /> <input type="reset" value="Reset" /></p>
    </form>
  </div>

</html>
```

The effect of the layout

- The visual effect and action is the same as for fragments without layout.
 - However, with more pages it is more convenient and natural.

[StudentList](#) [Home](#) [Seed](#) [Add new](#)

Students list

Id	First name	Last name	Index	Action		
1	Adam	Fox	12345	details	edit	remove
2	Ewa	Snake	12346	details	edit	remove
3	John	Brown	22222	details	edit	remove

25 października 2022 11:11:07 CEST

©Dariusz Konieczny

[StudentList](#) [Home](#) [Seed](#) [Add new](#)

Add a new student

Id:

First name:

Last name:

index:

login:

password:

©Dariusz Konieczny